1. What is Power Pivot?

Power Pivot is an Excel add-in that allows you to:

- Work with large datasets (millions of rows) easily.
- Create relationships between multiple tables (like a mini-database).
- Perform advanced calculations using DAX formulas.
- Build interactive reports and pivot tables from complex data.

Think of it as supercharged Pivot Tables with database-like capabilities.

---

2. Key Concepts

1. **Data Model:** Stores tables in a data model so you can use multiple tables together without merging them.
2. **Relationships:** Link tables using keys (like ProductID or CustomerID) instead of copying all data into one table.
3. **DAX (Data Analysis Expressions):** Formulas to calculate sums, averages, percentages, and more dynamically.

---

3. Examples with 2 Tables

Example 1: Sales by Product and Region

Tables:

Products Table

ProductID	ProductName	Category
101	Pen	Stationery
102	Notebook	Stationery
103	Calculator	Electronics

Sales Table

SaleID	ProductID	Region	Quantity	Price
1	101	North	10	5
2	102	South	20	20
3	103	North	5	200
4	101	South	15	5

Power Pivot Steps:

1. Load both tables into Power Pivot.

2. Create relationship: `Sales.ProductID → Products.ProductID`.

3. Pivot Table: - Rows: ProductName - Columns: Region - Values: Sum of Quantity

Result:

ProductName	North	South
Pen	10	15
Notebook	0	20
Calculator	5	0

Example 2: Revenue and Profit by Product

Example 2: Revenue and Profit by Product

Tables:

Products Table

ProductID	ProductName	Cost
101	Pen	2
102	Notebook	10

Sales Table

SaleID	ProductID	Quantity	Price
1	101	10	5
2	102	20	20

Power Pivot Steps: 1. Load tables into Data Model. 2. Create relationship: `Sales.ProductID → Products.ProductID`. 3. Create measures: - Revenue = `Quantity * Price` - Profit = `Revenue – (Quantity * Cost)`

Pivot Table Example:

ProductName	Revenue	Profit
Pen	50	30
Notebook	400	200

Tips for Class

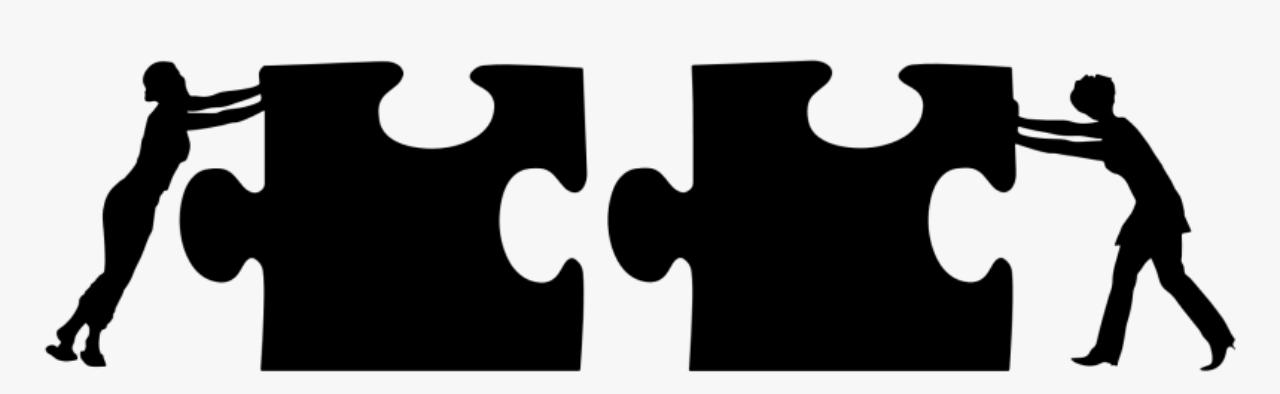
- Power Pivot handles **large datasets** efficiently.
- **Relationships** connect tables like puzzle pieces.
- Start with simple DAX formulas like SUM, AVERAGE before moving to complex ones.

TOPICS IN JOIN

- WHAT IS JOIN?
- USE OF JOIN
- JOIN TYPES
- WHICH JOIN TO USE
- JOIN SYNTAX
- EXAMPLES IN SQL

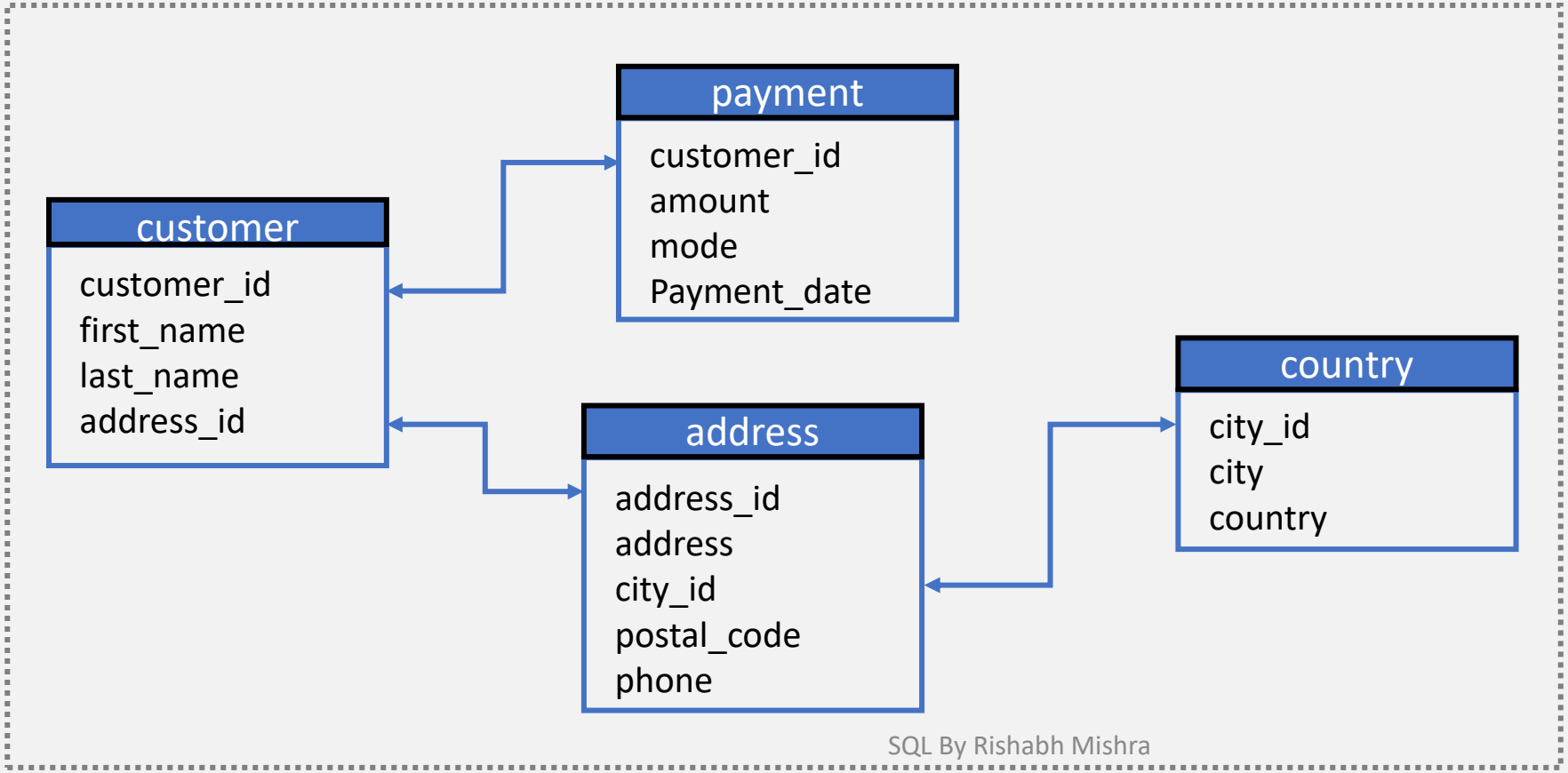
SQL JOIN

- **JOIN** means to combine something.
- A **JOIN** clause is used to combine data from two or more tables, based on a related column between them
- Let’s understand the joins through an example:



JOIN Example

Question: How much amount was paid by customer ‘Madan’, what was mode and payment date?



Database

JOIN Example

customer_id [PK] bigint	first_name character varying (50)	last_name character varying (50)	address_id bigint
1	Mary	Smith	5
2	Madan	Mohan	6
3	Linda	Williams	7
4	Barbara	Jones	8
5	Elizabeth	Brown	9

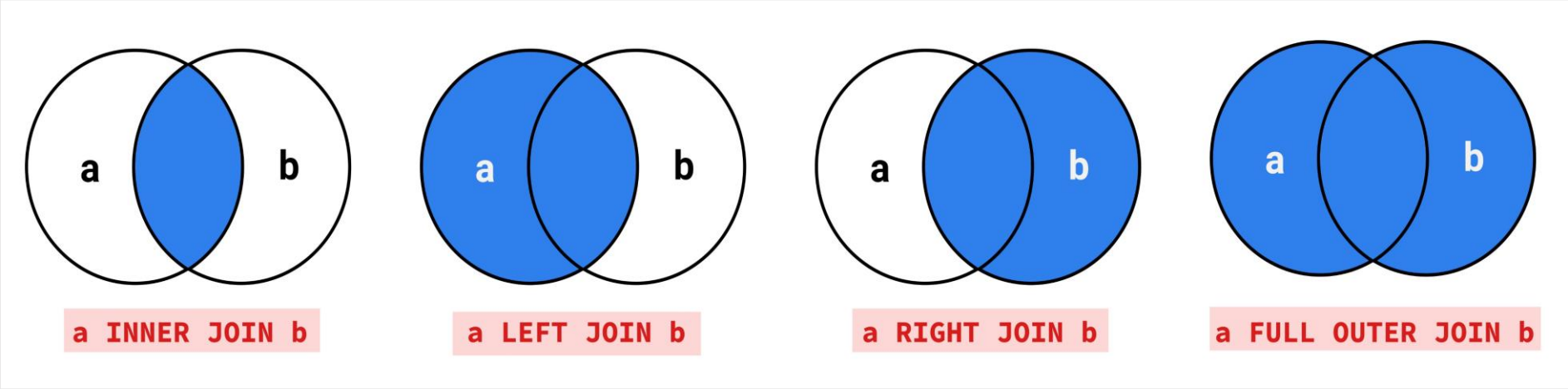
customer_id [PK] bigint	amount bigint	mode character varying (50)	payment_date date
1	60	Cash	2020-09-24
2	30	Credit Card	2020-04-27
3	90	Credit Card	2020-07-07
4	50	Debit Crad	2020-02-12
5	40	Mobile Payment	2020-11-20

Question: How much amount was paid by customer ‘Madan’, what was mode and payment date?

Answer: Amount = 30,
Mode = Credit Card,
Date = 2020-04-27

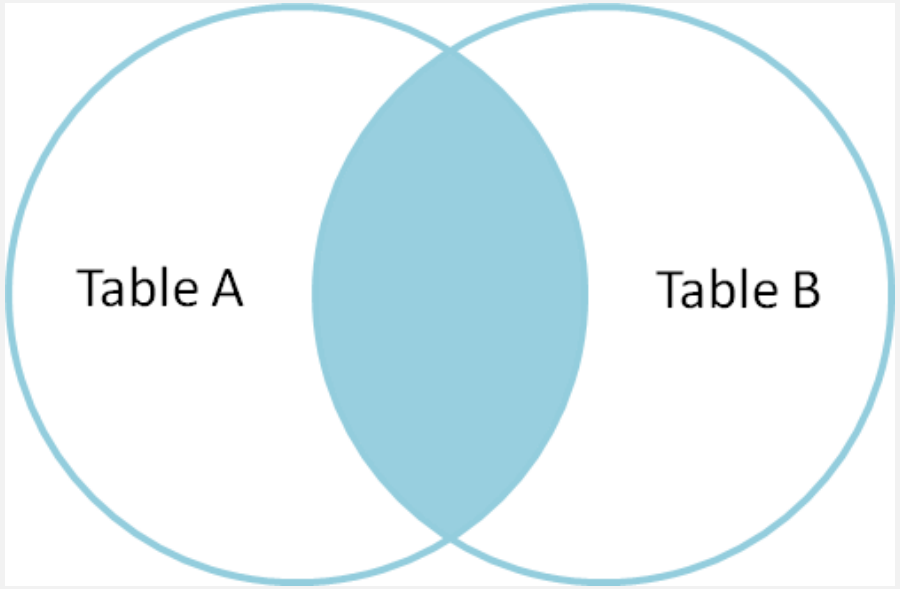
TYPES OF JOINS

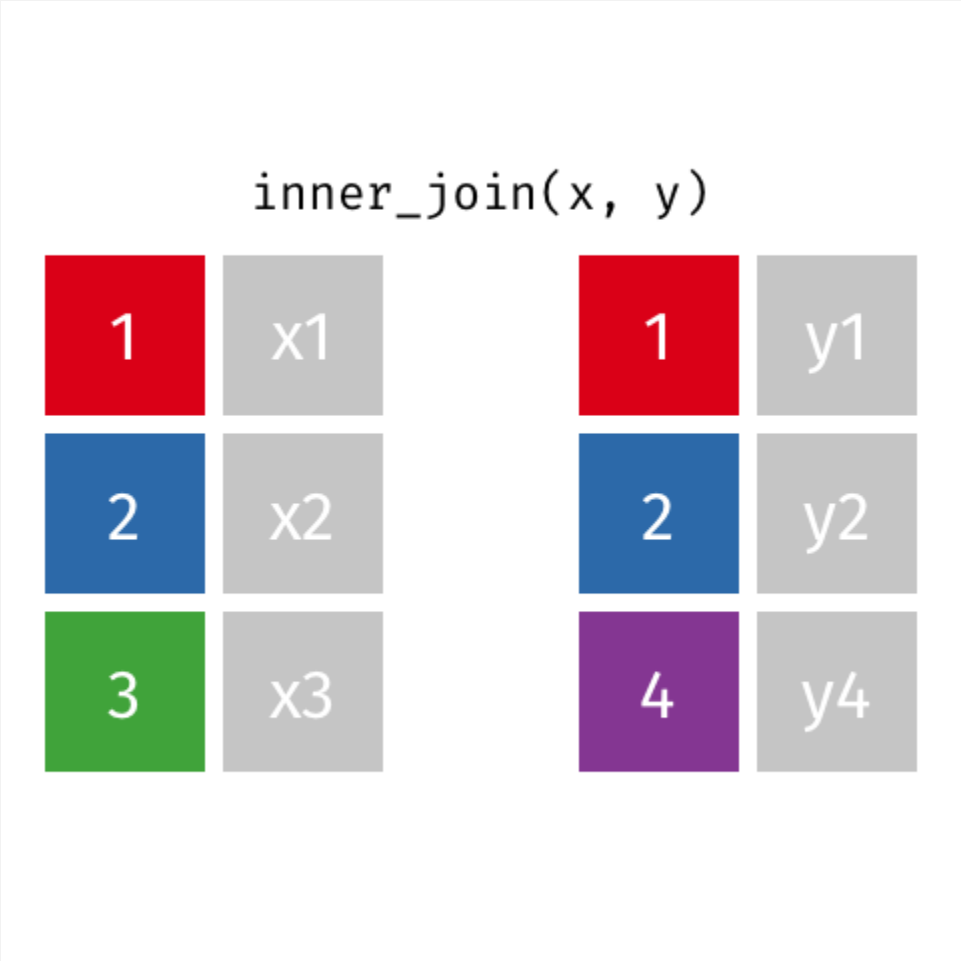
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN



INNER JOIN

- Returns records that have matching values in both tables

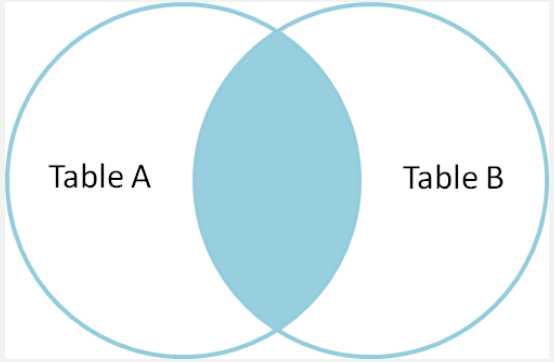




INNER JOIN

- **Syntax**

```
SELECT column_name(s)
FROM TableA
INNER JOIN TableB
ON TableA.col_name = TableB.col_name
```

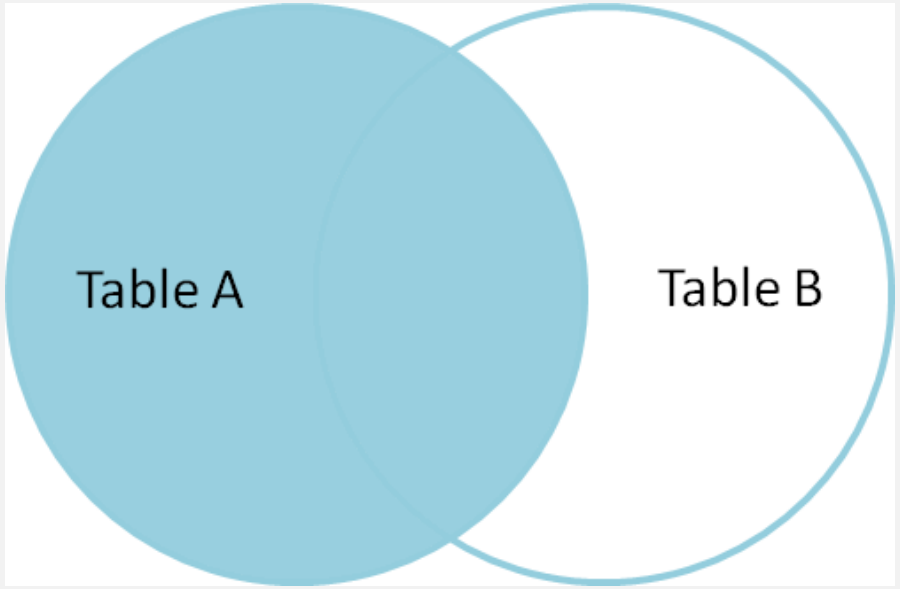


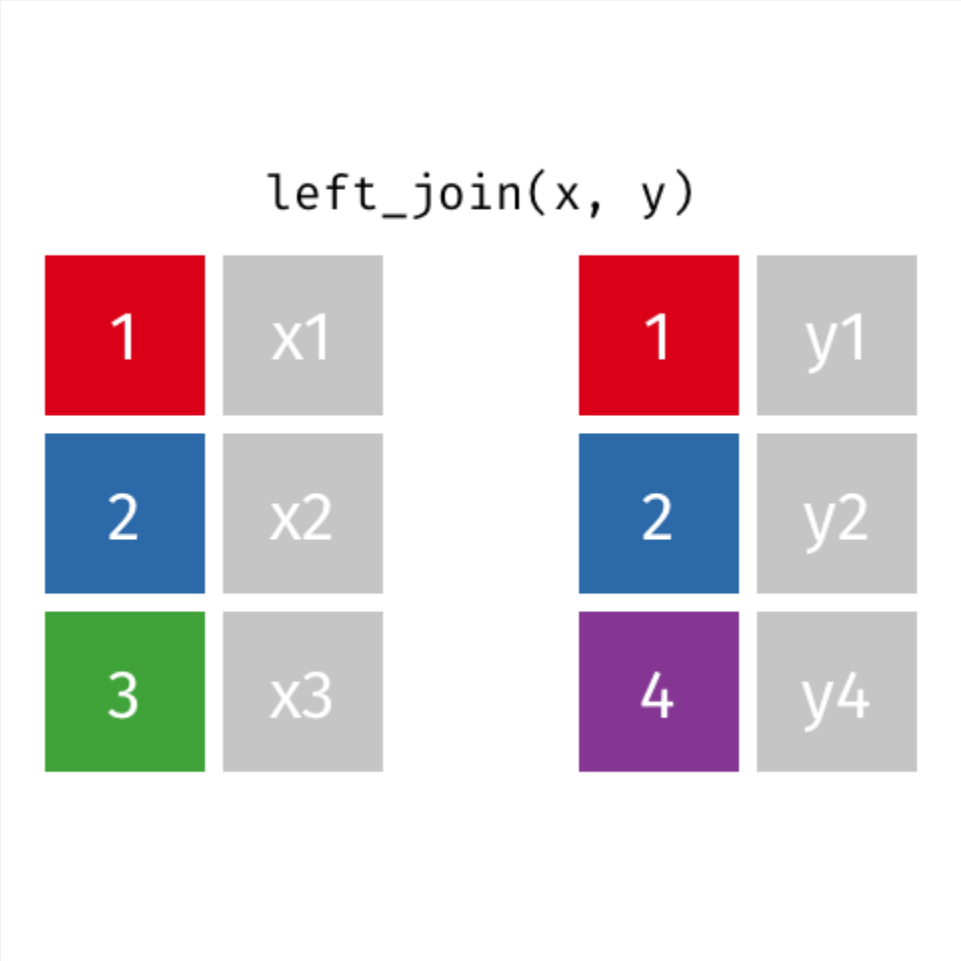
- **Example**

```
SELECT *
FROM customer AS c
INNER JOIN payment AS p
ON c.customer_id = p.customer_id
```

LEFT JOIN

- Returns all records from the left table, and the matched records from the right table

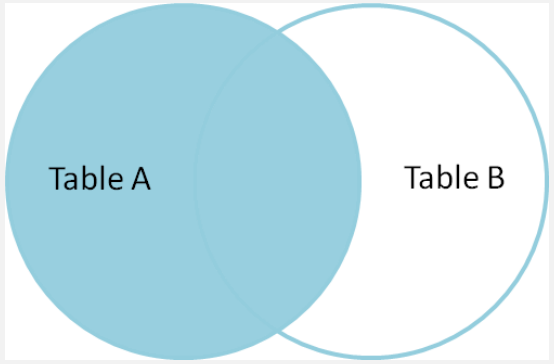




LEFT JOIN

- **Syntax**

```
SELECT column_name(s)
FROM TableA
LEFT JOIN TableB
ON TableA.col_name = TableB.col_name
```

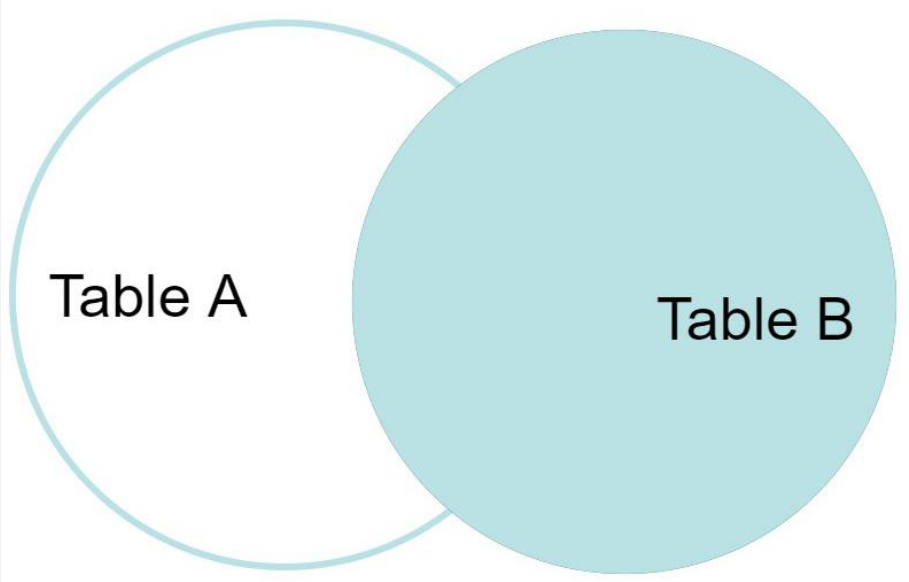


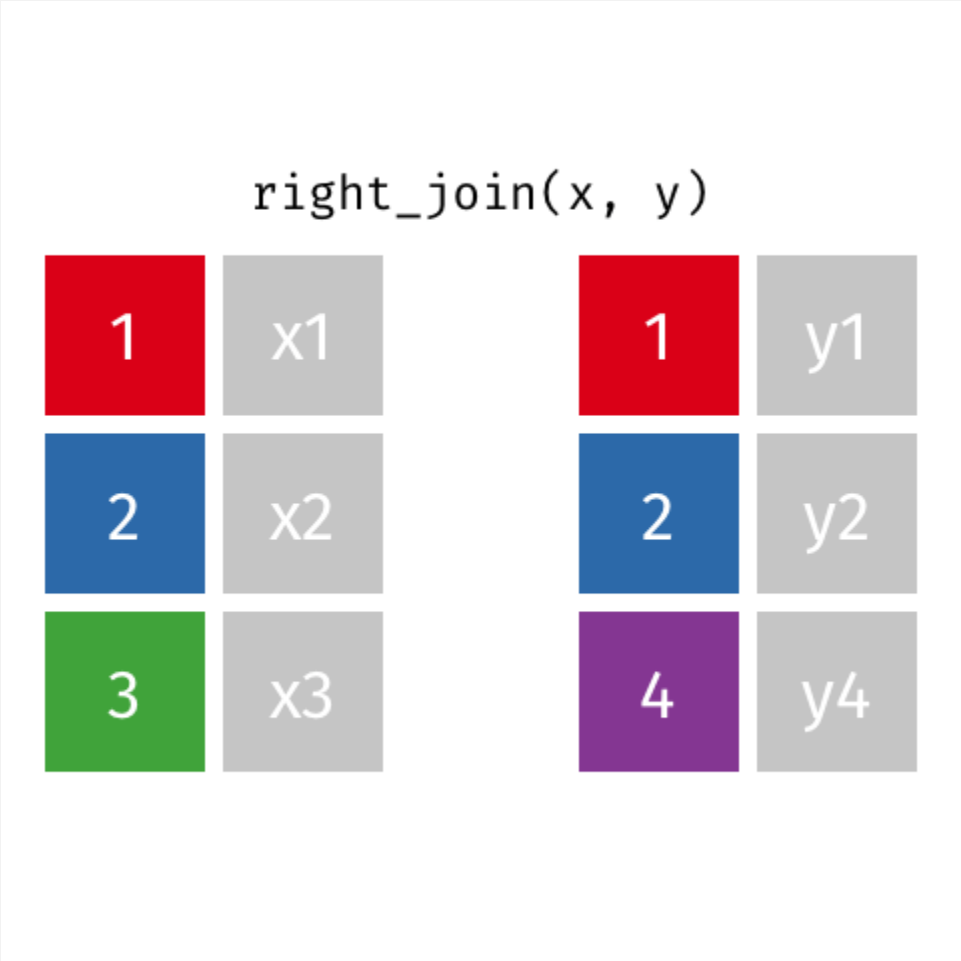
- **Example**

```
SELECT *
FROM customer AS c
LEFT JOIN payment AS p
ON c.customer_id = p.customer_id
```

RIGHT JOIN

- Returns all records from the right table, and the matched records from the left table

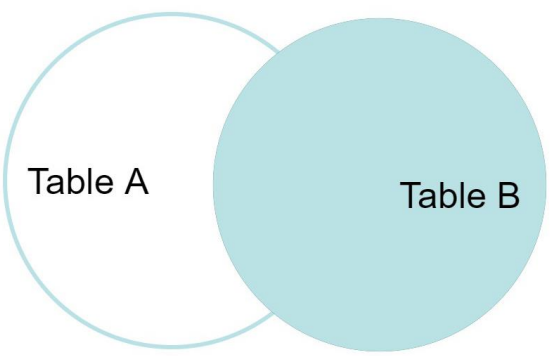




RIGHT JOIN

- **Syntax**

```
SELECT column_name(s)
FROM TableA
RIGHT JOIN TableB
ON TableA.col_name = TableB.col_name
```

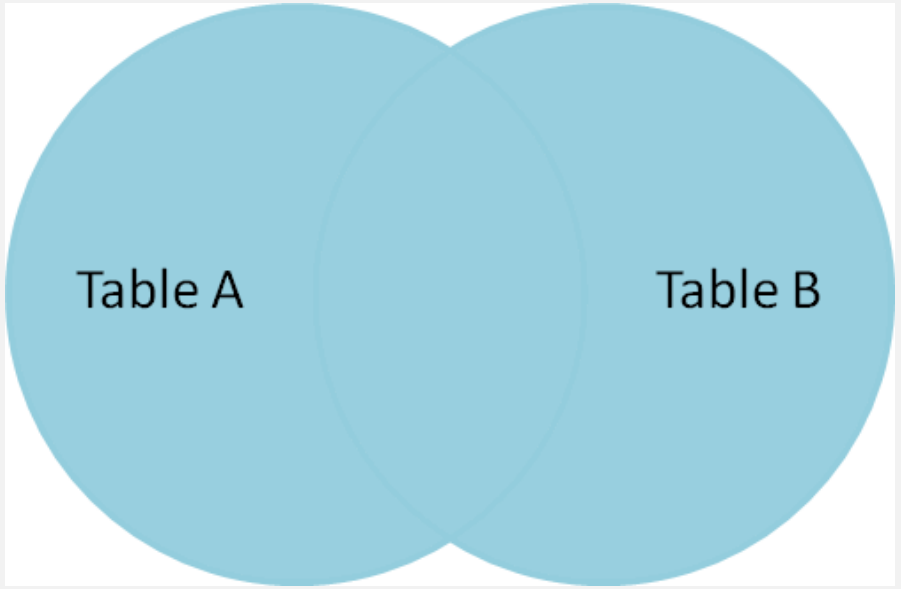


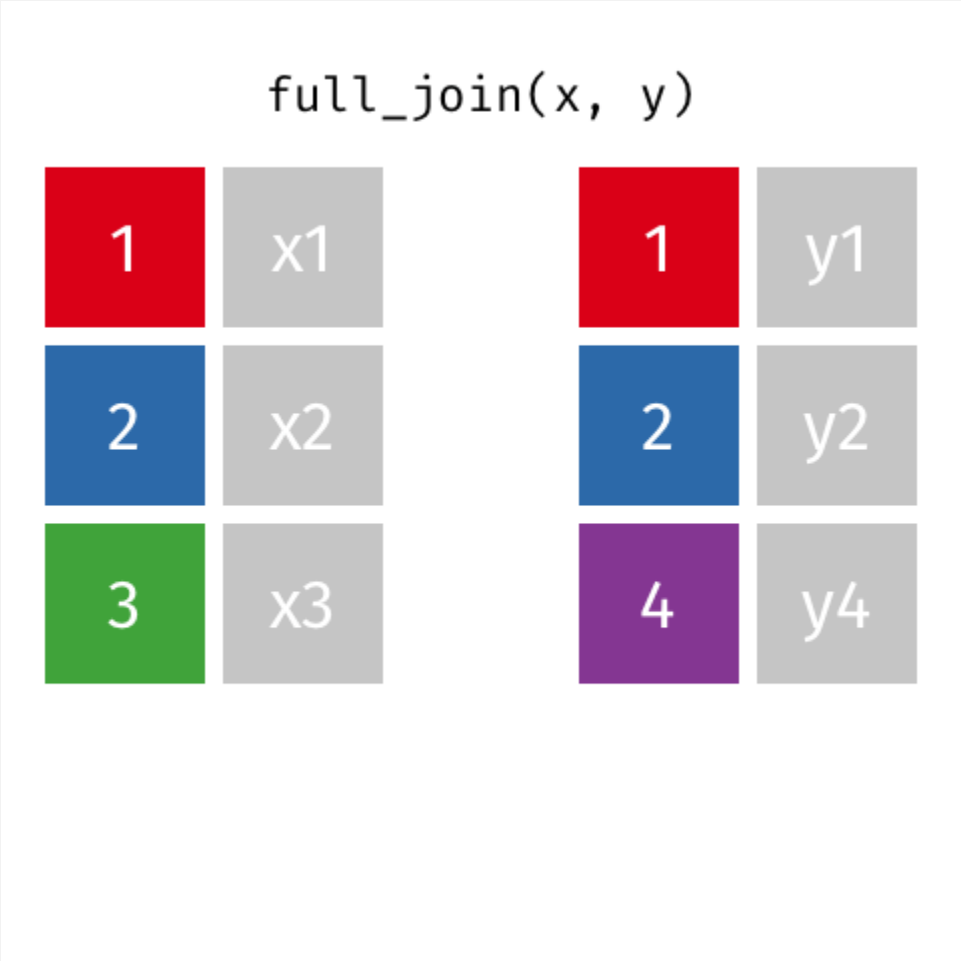
- **Example**

```
SELECT *
FROM customer AS c
RIGHT JOIN payment AS p
ON c.customer_id = p.customer_id
```

FULL JOIN

- Returns all records when there is a match in either left or right table

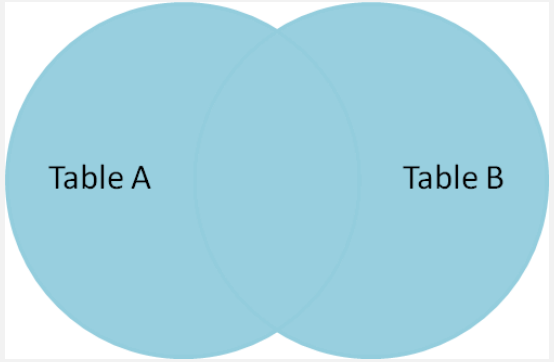




FULL JOIN

- **Syntax**

```
SELECT column_name(s)
FROM TableA
FULL OUTER JOIN TableB
ON TableA.col_name = TableB.col_name
```

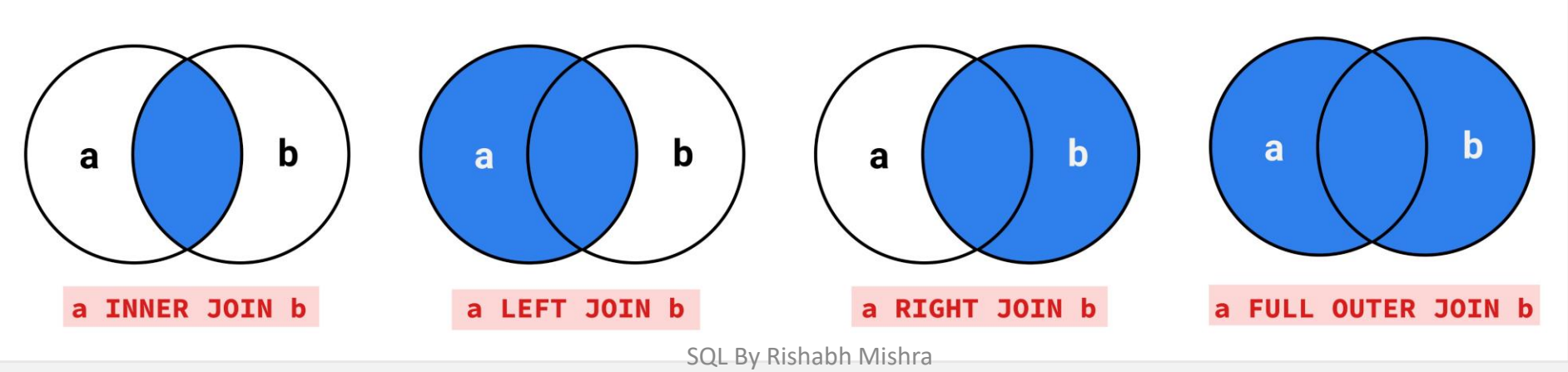


- **Example**

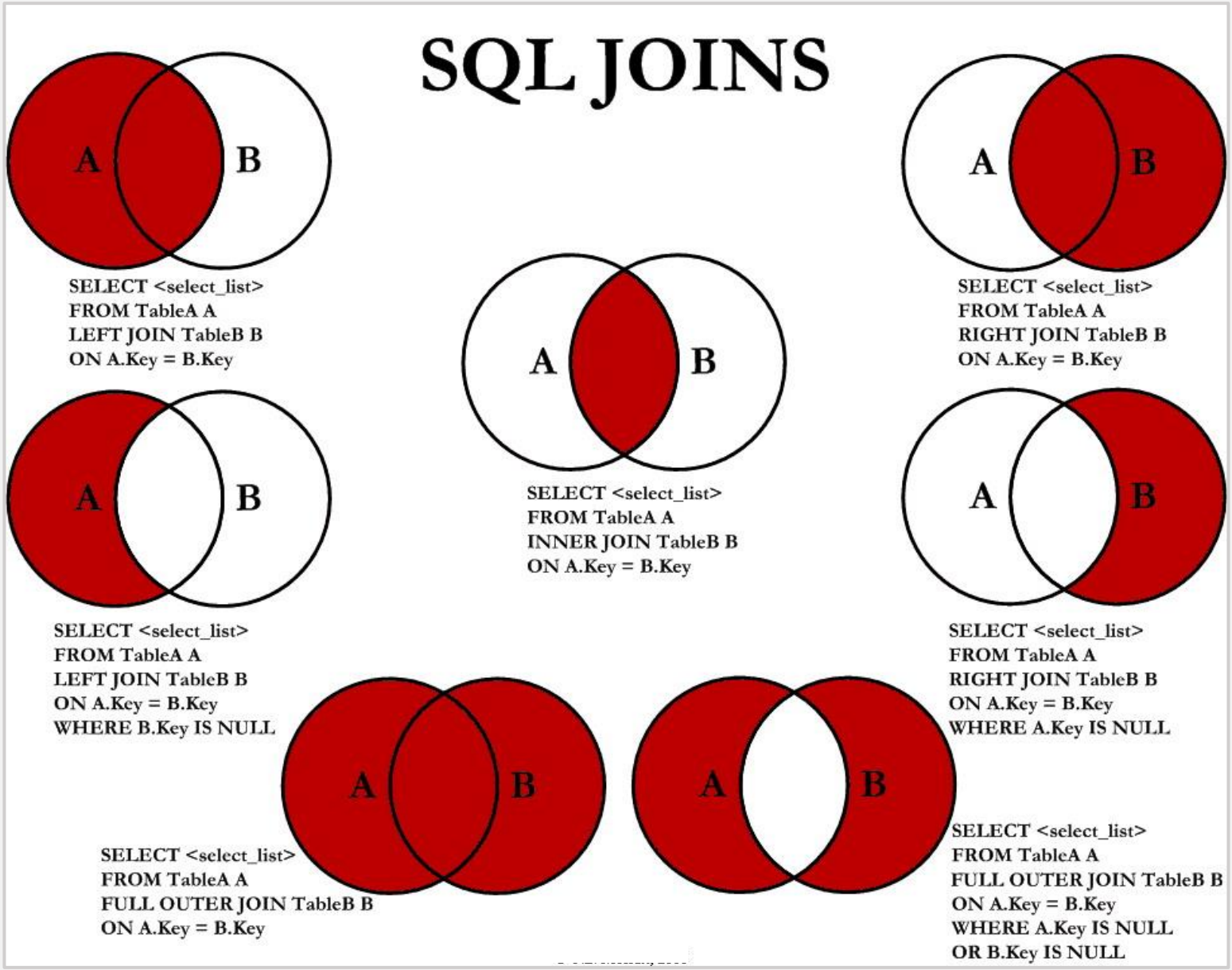
```
SELECT *
FROM customer AS c
FULL OUTER JOIN payment AS p
ON c.customer_id = p.customer_id
```

Which JOIN To Use

- **INNER JOIN:** Returns records that have matching values in both tables
- **LEFT JOIN:** Returns all records from the left table, and the matched records from the right table
- **RIGHT JOIN:** Returns all records from the right table, and the matched records from the left table
- **FULL JOIN:** Returns all records when there is a match in either left or right table



JOIN CHEAT SHEET



SELF JOIN

- A **self join** is a regular join in which a table is joined to itself
- **SELF Joins** are powerful for comparing values in a column of rows with the same table
- Syntax

```
SELECT column_name(s)
FROM Table AS T1
JOIN Table AS T2
ON T1.col_name = T2.col_name
```

SELF JOIN example

empid [PK] bigint	empname character varying (50)	manager_id bigint
1	Agni	3
2	Akash	4
3	Dharti	2
4	Vayu	3

Table: emp

- Find the name of respective managers for each of the employees?

SELF JOIN example

empid [PK] bigint	empname character varying (50)	manager_id bigint
1	Agni	3
2	Akash	4
3	Dharti	2
4	Vayu	3

mngr character varying (50)
Dharti
Vayu
Akash
Dharti

```
SELECT T2.empname, T1.empname
FROM emp AS T1
JOIN emp AS T2
ON T1.empid = T2.manager_id
```

UNION

The SQL **UNION** clause/operator is used to combine/concatenate the results of two or more SELECT statements without returning any duplicate rows and keeps **unique records**

To use this UNION clause, each SELECT statement must have

- The same number of columns selected and expressions
- The same data type and
- Have them in the same order
- **Syntax**

```
SELECT column_name(s) FROM TableA
UNION
SELECT column_name(s) FROM TableB
```

- **Example**

```
SELECT cust_name, cust_amount from custA
UNION
SELECT cust_name, cust_amount from custB
```

UNION ALL

In **UNION ALL** everything is same as **UNION**, it combines/concatenate two or more table but keeps all records, **including duplicates**

- **Syntax**

```
SELECT column_name(s) FROM TableA
UNION ALL
SELECT column_name(s) FROM TableB
```

- **Example**

```
SELECT cust_name, cust_amount from custA
UNION ALL
SELECT cust_name, cust_amount from custB
```

UNION Example

Table: custA

cust_name character (30)	cust_amount bigint
Madan Mohan	2100
Gopi Nath	1200
Govind Dev	5000

Table: custB

cust_name character (30)	cust_amount bigint
Gopal Bhat	1500
Madan Mohan	2100